



南开大学
Nankai University

南 开 大 学

数据安全实验报告

实验 1：半同态加密应用实践

学院：密码与网络空间安全学院

专业：物联网工程

学号：2310422

姓名：谢小珂

指导教师：刘哲理

2026 年 4 月 20 日

目录

1 实验要求	1
2 实验环境	1
2.1 安装 phe 库	1
2.2 验证环境	1
3 基础实验	2
3.1 基于 Python 的 phe 库完成加法和标量乘法的验证	2
3.1.1 实验代码	2
3.1.2 实验分析	3
3.2 基于 Python 的 phe 库完成隐私信息获取的功能	4
3.2.1 实验代码	4
3.2.2 实验分析	5
4 扩展实验	6
4.0.1 实验代码	6
4.0.2 实验分析	8
5 实验总结与心得体会	9

1 实验要求

1. 基于 Paillier 算法实现隐私信息获取: 从服务器给定的 m 个消息中获取其中一个, 不得向服务器泄露获取了哪一个消息, 同时客户端能完成获取消息的解密。
2. 扩展实验: 有能力的同学, 可以在客户端保存对称密钥 k , 在服务器端存储 m 个用对称密钥 k 加密的密文, 通过隐私信息获取方法得到指定密文后能解密得到对应的明文。

2 实验环境

安装 python 完毕, 打开控制台, 输入 python 命令, 会显示:

```
命令提示符 - python
Microsoft Windows [版本 10.0.26200.8246]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Xiexi>python
Python 3.12.6 (tags/v3.12.6:a4a2d2b, Sep 6 2024, 20:11:23) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

图 1: 实验环境安装

输入 python 程序: `from phe import paillier`

该命令将导入 phe 库的 paillier 功能, 第一次执行结果如图2:

```
命令提示符 - python
Microsoft Windows [版本 10.0.26200.8246]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\Xiexi>python
Python 3.12.6 (tags/v3.12.6:a4a2d2b, Sep 6 2024, 20:11:23) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> from phe import paillier
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'phe'
>>>
```

图 2: 未安装 phe 库

2.1 安装 phe 库

实验需要安装 **phe** 库, 以使用 **Paillier** 算法。

首先, 输入 `exit()` 退出当前 python 环境, 切回控制台模式。

输入 `pip install phe` 完成 phe 库的安装。

```
C:\Users\Xiexi>pip install phe
Collecting phe
  Downloading phe-1.5.0-py2.py3-none-any.whl.metadata (3.8 kB)
Downloading phe-1.5.0-py2.py3-none-any.whl (53 kB)
Installing collected packages: phe
Successfully installed phe-1.5.0

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

图 3: 安装 phe 库

2.2 验证环境

验证安装是否成功, 在命令行之中进入 python 环境, 然后输入如下命令: `from phe import paillier`

结果如图4: `import phe print(phe.__version__)` 可以查看 phe 库的版本。

```
C:\Users\xiexi>python
Python 3.12.6 (tags/v3.12.6:a4a2d2b, Sep 6 2024, 20:11:23) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> from phe import paillier
>>> import phe
>>> e库是1.5.0版本, 这已经是现在最新的版
```

图 4: 验证环境查看版本

3 基础实验

3.1 基于 Python 的 phe 库完成加法和标量乘法的验证

3.1.1 实验代码

```
1 from phe import paillier # 开源库
2 import time # 做性能测试
3
4 ##### 设置参数
5 print("默认私钥大小: ", paillier.DEFAULT_KEYSIZE)
6 #生成公私钥
7 public_key, private_key = paillier.generate_paillier_keypair()
8 # 测试需要加密的数据
9 message_list = [3.1415926,100,-4.6e-12]
10
11 ##### 加密操作
12 time_start_enc = time.time()
13 encrypted_message_list = [public_key.encrypt(m) for m in message_list]
14 time_end_enc = time.time()
15 print("加密耗时s: ",time_end_enc-time_start_enc)
16 print("加密数据 (3.1415926) :",encrypted_message_list[0].ciphertext())
17
18 ##### 解密操作
19 time_start_dec = time.time()
20 decrypted_message_list = [private_key.decrypt(c) for c in encrypted_message_list]
21 time_end_dec = time.time()
22 print("解密耗时s: ",time_end_dec-time_start_dec)
23 print("原始数据 (3.1415926) :",decrypted_message_list[0])
24
25 ##### 测试加法和乘法同态
26 a,b,c = encrypted_message_list # a,b,c分别为对应密文
27 a_sum = a + 5 # 密文加明文, 已经重载了+运算符
28 a_sub = a - 3 # 密文加明文的相反数, 已经重载了-运算符
29 b_mul = b * 6 # 密文乘明文,数乘
30 c_div = c / -10.0 # 密文乘明文的倒数
31
32 print("a+5 密文:",a.ciphertext()) # 密文纯文本形式
33 print("a+5=",private_key.decrypt(a_sum))
34 print("a-3",private_key.decrypt(a_sub))
35 print("b*6=",private_key.decrypt(b_mul))
36 print("c/-10.0=",private_key.decrypt(c_div))
37
38 ##密文加密文
39 print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a+b))
40 #报错, 不支持a*b, 即两个密文直接相乘
41 #print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a*b))
```

结果如图6所示:

3.2 基于 Python 的 phe 库完成隐私信息获取的功能

服务器端拥有多个数值，要求客户端能基于 Paillier 实现从服务器读取一个指定的数值并正确解密，但服务器不知道所读取的是哪一个

首先，我们要基于 Paillier 协议进行设计。

对 Paillier 的标量乘的性质进行扩展，我们知道：

1. 数值“0”的密文与任意数值的标量乘也是 0；
2. 数值“1”的密文与任意数值的标量乘将是数值本身。

数学表达：

$$Dec(Enc(m_1)^{m_2} \bmod n^2) = m_1 \cdot m_2$$

在代码层面，phe 库将密文与明文的乘法运算符 (*) 重载为执行此同态运算。

```

服务器端：产生数据列表data_list={m1, m2, ..., mn}
客户端：
· 设置要选择的数据位置为pos
· 生成选择向量select_list={0,...,1,..., 0}，其中，仅有pos的位置为1
· 生成密文向量enc_list={E(0),..., E(1),..., E(0)}
· 发送密文向量enc_list给服务器
服务器端：
· 将数据与对应的向量相乘后累加得到密文
    c=m1*enc_list[1]+...+mn*enc_list[n]
· 返回密文c给客户端
客户端：解密密文c得到想要的结果

```

图 6: 实验设计

3.2.1 实验代码

```

1 from phe import paillier # 开源库
2 import random # 选择随机数
3
4 ##### 设置参数
5 # 服务器端保存的数值
6 message_list = [100,200,300,400,500,600,700,800,900,1000]
7 length = len(message_list)
8 # 客户端生成公私钥
9 public_key, private_key = paillier.generate_paillier_keypair()
10 # 客户端随机选择一个要读的位置
11 pos = random.randint(0,length-1)
12 print("要读起的数值位置为：",pos)
13
14 ##### 客户端生成密文选择向量
15 select_list=[]
16 enc_list=[]
17 for i in range(length):
18     select_list.append( i == pos )
19     enc_list.append( public_key.encrypt(select_list[i]) )
20
21 # for element in select_list:
22 # print(element)

```

```

23 # for element in enc_list:
24 # print(private_key.decrypt(element))
25
26 ##### 服务器端进行运算
27 c=0
28 for i in range(length):
29     c = c + message_list[i] * enc_list[i]
30 print("产生密文: ",c.ciphertext())
31
32 ##### 客户端进行解密
33 m=private_key.decrypt(c)
34 print("得到数值: ",m)

```

结果如图7所示:

```

Python 3.12.6 (tags/v3.12.6:a4a2d2b, Sep 6 2024, 20:11:23) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: K:/大三下/数据安全/实验一二.py
要读取的数值位置为: 2
产生密文:  208156727103272091753488063381060049108670603218491010627906676648447918257454461212434170
284739900934735998523970188466786298249754109320852200060207547502642032054275817715037437
218425066802016942391949552088757511686580003026241265692563507686174555160692373602074331
089836034950541620588755947504032829800527586076989025087967180876451570045552836839955534
985564002925009751370624316519973456446690528403080038553386917873588534357310947296244199
283375248132368080237696760891652422908074321787474578395737406799022443436737542417151867
019452272203313682774929905386790651771869068017145623169860613363904649637524588411469188
584249369798934029988758291613696500109093836597188580133301754796283795938162235520528521
95036555821568492015060631289652154921159614979561518033838630844557130622499429770187375
959405855204147741549624337626716932541916639724903054170532435493229539636879342607696048
751644311684208239999071938790486654277850056587898427544450923186264695048417137014369782
404281963801501915639311477667991604425610910304869592755177978123134313756440399177290833
690798968256127179164355348746800739176169903088742575606770257613686632208068951375340638
054592032796825865215710221637988871424275882993928658255232319597315856033758108952030382
56378754700739353301982929246295217006686242763896046259868208829220723832709484551376857
906279384876214894054966628250887947295746113705760995312134785550650344610991801148357085
215438549627388884681405509548884512830808085244328706210251672085535089957366099398097375
27309521893113758966335860440993581150148977713539923410704870269912845775458707370764175
084419391613816761130718745256027023112121206060386048321025157464895068177853027215928190
497691739823769295463771660984869345318317676453699645610093
得到数值:  300
>>>

```

图 7: 实验输出结果

3.2.2 实验分析

实验代码模拟了客户端与服务器之间的交互过程，具体分为以下四个阶段：

1. 参数初始化与密钥生成：

服务器端维护一个消息列表 `message_list` (如 `[100, 200, ..., 1000]`)。

客户端通过 `paillier.generate_paillier_keypair()` 生成自己的公钥 `public_key` 和私钥 `private_key`。

2. 客户端生成密文选择向量：

客户端确定要读取的位置 `pos`。客户端构造一个“选择向量” `select_list`：只有 `pos` 位置的值为 1，其余位置均为 0。

核心加密：客户端使用公钥对该向量进行逐位加密，生成 `enc_list`。由于 Paillier 具有语义安全性，服务器即使拿到这一组密文，也无法分辨哪一个是 1，哪一个是 0。

3. 服务器端计算：

服务器收到 `enc_list` 后，将其与本地的明文数据 `message_list` 进行同态运算。计算公式为：

$$c = \sum (\text{message_list}[i] \times \text{enc_list}[i])$$

逻辑解释: 根据前述性质, 当 $\text{select_list}[i] = 0$ 时, 结果为 0 的密文; 当 $\text{select_list}[i] = 1$ 时, 结果为该位置原始数值的密文。通过密文加法 (代码中的 $+$), 服务器最终得到一个汇总密文 c , 它实际上就是目标数值的加密版本。

4. 客户端解密:

客户端收到结果 c 后, 使用自己的私钥进行解密 $\text{private_key.decrypt}(c)$, 从而获得所需数值。

4 扩展实验

实验背景与问题建模 基础实验中服务器存储明文数据, 仅通过同态加密保护客户端查询位置隐私。在实际隐私计算场景中, 如安全外包计算, 服务器通常被视为诚实但好奇的第三方。本实验目标是实现双重隐私保护, 在保护查询位置的同时, 进一步保护服务器上的静态数据。数据内容隐私要求服务器无法获知存储内容, 通过对称加密实现; 查询行为隐私要求服务器无法获知客户端访问位置, 通过 Paillier 同态加密实现。

核心解题思路 解题思路基于 Paillier 加法同态与标量乘法特性展开。对称加密层负责数据静态防护, 客户端生成对称密钥 k , 采用对称加密算法将原始数据 m 转换为密文 $c = m \oplus k$, 服务器仅存储加密后的数据, 无法获取原始信息。Paillier 同态层负责动态计算保护, 客户端生成加密选择向量, 目标索引位置为 $E(1)$, 其余位置为 $E(0)$ 。

盲计算逻辑依托标量乘法特性 $a \cdot E(m) = E(a \cdot m)$ 实现。服务器执行加权求和计算 $C_{final} = \sum_{i=1}^n (c_i \cdot E(v_i))$ 。当 $i = pos$ 时, $c_i \cdot E(1) = E(c_i)$; 当 $i \neq pos$ 时, $c_i \cdot E(0) = E(0)$ 。结合加法同态性质, 最终求和结果解密后即为目标位置的对称密文 c_{pos} 。

实验流程设计 实验流程分为四个阶段。初始化阶段, 客户端本地保存 Paillier 私钥 sk 与对称密钥 k , 将公钥 pk 与对称加密后的数据列表上传至服务器。查询构造阶段, 客户端根据目标索引生成由 $E(0)$ 和 $E(1)$ 组成的加密向量并发送至服务器。云端处理阶段, 服务器在不接触私钥与对称密钥的前提下, 完成同态加权求和计算, 返回单一 Paillier 密文。两级解密阶段, 客户端先使用 sk 解密得到对称密文, 再使用 k 解密恢复原始明文数据。

4.0.1 实验代码

```

1 import random
2 from phe import paillier
3
4 # 1. 定义加解密逻辑
5 # 模拟对称加密 (客户端和服务器约定的逻辑)
6 # 这里使用简单的异或运算, 因为它既能处理数值, 也满足实验演示需求
7 def sym_encrypt(plain, key):
8     return plain ^ key
9 def sym_decrypt(cipher, key):
10    return cipher ^ key
11
12 # 2. 准备阶段 (客户端与服务器)
13 # 客户端: 保存对称密钥 k
14 k = 20231024
15 # 原始明文数据列表
16 raw_data = [100, 200, 300, 400, 500, 600, 700]
17 # 服务器: 存储加密后的密文

```

```

18 # 在服务器眼中, 这些只是普通的整数, 无法得知原始数据
19 server_storage = [sym_encrypt(x, k) for x in raw_data]
20 print("-" * 50)
21 print(f"服务器存储的对称密文列表:\n{server_storage}")
22 print("-" * 50)
23
24 # 3. 客户端生成请求 (Paillier加密)
25 # 客户端生成 Paillier 公私钥
26 public_key, private_key = paillier.generate_paillier_keypair()
27 # 客户端确定要查询的位置 (例如索引 3, 对应原始数据 400)
28 target_index = 3
29 print(f"客户端: 我要查询位置为 {target_index} 的数据。")
30 # 客户端生成加密的选择向量 (仅查询位置为 1, 其余为 0)
31 enc_select_vector = []
32 for i in range(len(server_storage)):
33     val = 1 if i == target_index else 0
34     # 对 0 或 1 进行 Paillier 加密
35     enc_select_vector.append(public_key.encrypt(val))
36 # 展示其中一个 Paillier 密文的样子
37 sample_ciphertext = str(enc_select_vector[0].ciphertext())
38 print(f"\n客户端发送的选择向量 (部分展示第一个元素的密文): \n{sample_ciphertext[:100]}... [此处省略上
    千位]")
39
40 # 4. 服务器端同态运算
41 # 服务器根据客户端发来的加密向量和自己存储的对称密文进行运算
42 # homo_result =  $\sum (server\_storage[i] * enc\_select\_vector[i])$ 
43 homo_result = 0
44 for i in range(len(server_storage)):
45     # 标量乘法: 明文数值 * Paillier密文
46     term = server_storage[i] * enc_select_vector[i]
47     homo_result = homo_result + term
48 # 此时 homo_result 是一个 Paillier 密文对象
49 print("\n服务器: 同态计算完成, 已生成加密结果。")
50 full_cipher_str = str(homo_result.ciphertext())
51 print(f"服务器传回的【结果密文】(前100位):\n{full_cipher_str[:100]}...")
52 print(f"该密文总长度为: {len(full_cipher_str)} 位数字")
53
54 # 5. 客户端最终解密
55 # 第一步: 客户端用 Paillier 私钥解开第一层 (同态层)
56 # 得到的结果应该是对应位置的对称密文
57 decrypted_sym_cipher = private_key.decrypt(homo_result)
58 print("-" * 50)
59 print(f"客户端第一步解密 (Paillier解密) 结果: {decrypted_sym_cipher}")
60 # 第二步: 客户端用对称密钥 k 解开第二层 (原始数据层)
61 final_plain_text = sym_decrypt(decrypted_sym_cipher, k)
62 print(f"客户端第二步解密 (对称密钥解密) 结果: {final_plain_text}")
63 # 验证
64 if final_plain_text == raw_data[target_index]:
65     print("\n[结论]: 实验成功!")
66     print("服务器全程不知道查询位置 (因为选择向量是加密的)。")
67     print("服务器全程不知道原始明文 (因为存储的是对称加密后的结果)。")

```

基于 Python 的 phe 库实现, 采用客户端-服务器交互模型, 结合对称加密与 Paillier 半同态加密, 本实验核心逻辑分为四大模块:

对称加密模块 通过 `sym_encrypt` 与 `sym_decrypt` 函数实现数据静态保护。客户端在数据上传前，使用对称密钥 k 对明文做异或加密。该方式高效处理批量数据，降低同态加密存储开销，保证服务器仅存储不可读密文。

密钥生成与加密请求构造 客户端调用 Paillier 算法生成公私钥对，并构建加密选择向量：目标索引位置为 $E(1)$ ，其余位置为 $E(0)$ 。依托语义安全性，相同明文加密结果随机，服务器无法推断查询位置，实现查询隐私保护。

云端同态计算模块 服务器执行加权求和运算 $C = \sum_{i=1}^m (Server_Storage[i] \cdot Enc_Vector[i])$ 。基于标量乘法与加法同态性质，运算后仅保留目标位置对称密文的加密结果，全程无需解密，完成盲查询计算。

二重解密模块 客户端执行两级解密：首先通过私钥解密 Paillier 密文，得到对称密文；再利用对称密钥解密，恢复原始明文。数据在传输与存储阶段全程加密，仅客户端可获取明文，实现双重隐私保护。结果如图8所示：

```
服务器存储的对称密文列表:
[20230932, 20231096, 20230748, 20230880, 20230788, 20230440, 20230604]

客户端: 我要查询位置为 3 的数据。

客户端发送的选择向量 (部分展示第一个元素的密文):
25171159178494557877343837038361359621407629451768027981224146428519876283633953
17985105247921240003... [此处省略上千位]

服务器: 同态计算完成, 已生成加密结果。
服务器传回的【结果密文】(前100位):
15233074491992465835695165473914476048238678871117190723010392689431253609299369
42462414703496861133...
该密文总长度为: 1849 位数字

客户端第一步解密 (Paillier解密) 结果: 20230880
客户端第二步解密 (对称密钥解密) 结果: 400

[结论]: 实验成功!
服务器全程不知道查询位置 (因为选择向量是加密的)。
服务器全程不知道原始明文 (因为存储的是对称加密后的结果)。
```

图 8: 扩展实验输出结果

4.0.2 实验分析

本实验设定客户端查询索引为 3 的数据，对应原始明文为 400。在实验过程中，服务器端存储的对称密文列表为 [20230932, 20231096, 20230748, 20230880, ...]。当客户端发起请求时，服务器通过同态运算返回了一个长度为 1849 位的 Paillier 密文。

实验从三个维度验证了隐私信息获取 (PIR) 的有效性：

- **数据机密性**：服务器存储的对称密文（如索引 3 处的 20230880）与原始明文 400 无直接数值关联。在没有对称密钥 k 的情况下，服务器无法破解底层数据，保证了存储安全性。
- **查询位置隐私性**：观察实验输出，客户端发送的选择向量及服务器返回的结果均表现为超长位数的随机化数字（1849 位）。基于 Paillier 的语义安全性，服务器无法通过密文特征推断出 $E(1)$ 所在的位置，从而保护了客户端的检索意图。
- **同态运算正确性**：客户端通过二重解密过程（Paillier 解密 $\rightarrow$ 对称解密）精准还原出明文 400。这一结果印证了服务器端执行的加权求和运算 $\sum (c_i \cdot E(v_i))$ 在数学逻辑上的无损性。

本扩展实验通过“对称 + 同态”的混合加密架构，在不泄露数据内容与查询位置的前提下完成了安全检索，成功实现了隐私计算中“数据可用不可见”的核心目标。

5 实验总结与心得体会

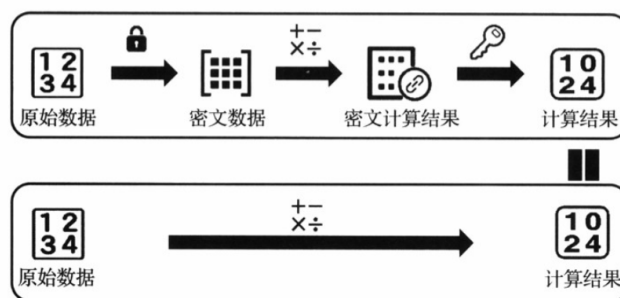


图 9: 同态加密的工作流程

本次实验通过 Python 的 ‘phe’ 库成功实现了基于 Paillier 方案的隐私信息获取系统。实验结合了对称加密的高效性与半同态加密的代数特性，如图 9所示，在密文域执行标量乘法与加法同态运算，实现了服务器在“双盲”状态下（即不知晓存储内容、不知晓查询位置）的数据检索。参考 PPT 中关于安全外包计算的模型，本实验通过“对称 + 同态”的混合架构，确保了数据在存储、传输及计算全生命周期内的机密性。实测结果显示，客户端能够从 1849 位的超长密文结果中精准还原原始明文，圆满达成了隐私计算中“数据可用不可见”的核心目标。